

Reranking in RAG

Retrieval-Augmented Generation

From first principles to production — a complete guide

01 What is RAG?
02 The Retrieval Problem
03 What is Reranking?
04 How Rerankers Work
05 Bi-encoder vs Cross-encoder
06 Popular Reranking Models
07 Reranking Pipeline
08 Code Examples
09 Metrics & Evaluation
10 Best Practices

01 What is RAG?

Retrieval-Augmented Generation (RAG) is an architecture that enhances Large Language Models by supplying them with external knowledge at inference time. Instead of relying solely on what the model learned during training, RAG **retrieves relevant documents** from a knowledge base and feeds them as context to the LLM before generating a response.

Step	Action	Description
1	Index	Chunk documents → embed with a bi-encoder → store in a vector DB
2	Retrieve	Embed query → ANN search → return top-k chunks
3	Augment	Prepend retrieved chunks to the prompt as context
4	Generate	LLM produces an answer grounded in retrieved context

02 The Retrieval Problem

Standard vector retrieval is powerful but imprecise. Embedding models compress an entire sentence or paragraph into a single fixed-size vector. When you query this space, you get documents that are *semantically nearby* — but nearby does not always mean *most relevant*.

Common retrieval failures

- Lexical gap — the query uses different words than the document, lowering cosine similarity
- Granularity mismatch — a short query vs a long chunk causes uneven embeddings
- False positives — topically related but factually irrelevant chunks sneak into top-k
- Ordering noise — the most relevant chunk may be ranked 8th, not 1st

■ Why this matters for generation

LLMs are sensitive to context position and quality. Irrelevant chunks dilute the signal, waste context-window tokens, and can cause hallucinations. Getting the right chunks to the top is as important as retrieving them at all.

03 What is Reranking?

Reranking is a **post-retrieval refinement step** that takes the initial candidate set from vector search and rescores every chunk against the query using a more powerful model. The result is a much tighter, higher-quality shortlist delivered to the LLM.

Stage	Model Type	Speed	Accuracy	Scale
Vector Search	Bi-encoder	Very Fast	Good	Millions
Reranking	Cross-encoder	Moderate	Excellent	Top 50–200
LLM Generation	Generative LLM	Slow	—	Top 3–10

The shaded row shows reranking's position in the pipeline — after fast retrieval, before expensive generation.

04 How Rerankers Work

A reranker is typically a **cross-encoder transformer**. Unlike bi-encoders that embed query and document independently, a cross-encoder receives the query and a candidate document *concatenated together* as a single input and outputs a relevance score.

Input format to the cross-encoder:

[CLS] <query> [SEP] <document chunk> [SEP]

The model attends **across both** query tokens and document tokens simultaneously, capturing fine-grained interactions (e.g., whether a number in the document matches a number asked in the query). This is computationally heavier than bi-encoding but produces significantly more accurate relevance judgements.

Why cross-attention matters

- Token-level alignment — 'What was the revenue in 2023?' can match '2023 revenue: \$4.2B' precisely
- Negation handling — cross-attention distinguishes 'X causes Y' from 'X does not cause Y'
- Entity disambiguation — the model resolves which 'Python' (snake vs language) fits the query
- Numerical reasoning — comparison between numbers in query and document is native

05 Bi-encoder vs Cross-encoder

Property	Bi-encoder	Cross-encoder
Inputs processed	Separately	Together (concatenated)
Attention scope	Within each text	Across query + document
Output	Two vectors → cosine sim	Single relevance score
Indexing	Pre-compute doc vectors	Cannot pre-compute
Inference latency	Sub-millisecond (lookup)	10–100ms per pair
Recall	High	Not applicable (no retrieval)
Precision	Moderate	Very high
Best use case	First-stage retrieval	Reranking top-k

■ Use both, not one or the other

The two-stage pipeline is complementary: bi-encoders give you speed and recall at scale, cross-encoders give you precision on the shortlist. Running a cross-encoder on a million documents would be prohibitively slow — that's why you first filter to ~50–200 candidates.

06 Popular Reranking Models

Model / API	Type	Notes
Cohere Rerank v3	API (commercial)	Best-in-class accuracy, multilingual, easy integration
bge-reranker-v2-m3	Open-source	BAAI model, strong multilingual performance, HuggingFace
ms-marco-MiniLM-L-6-v2	Open-source	Lightweight, fast, good for English RAG
ColBERT v2	Open-source	Late interaction; faster than full cross-encoder
FlashRank	Open-source	CPU-friendly, sub-second latency, plug-and-play
Jina Reranker v2	Open-source / API	8192 token context, code-aware
LLM-as-reranker	Prompting	Use an LLM to score relevance — flexible but expensive

07 Complete Reranking Pipeline

A production RAG pipeline with reranking follows these stages:

1. Ingestion

- Split documents into overlapping chunks (e.g., 512 tokens, 64-token overlap)
- Embed each chunk with a bi-encoder (e.g., text-embedding-3-small)
- Store vectors + metadata in a vector DB (Pinecone, Weaviate, pgvector, Chroma)

2. Query time — Retrieval

- Embed the user query with the same bi-encoder
- Run approximate nearest-neighbour (ANN) search to fetch top-50 candidates
- This step must be fast: target < 50ms

3. Query time — Reranking

- Pass (query, chunk) pairs to the cross-encoder reranker
- Reranker outputs a float relevance score per chunk
- Sort descending, keep top-5 (typical) for the LLM

4. Generation

- Assemble a prompt: system message + retrieved top-k as context + user query
- LLM generates an answer grounded in the retrieved evidence
- Optionally cite source documents in the response

08 Code Examples

Using Cohere Rerank API

cohere-python SDK — requires a Cohere API key:

```
import cohere co = cohere.Client("YOUR_COHERE_API_KEY") query = "What causes transformer attention to be quadratic?" documents = [ "Self-attention computes pairwise interactions between all tokens.", "Transformers were introduced in 'Attention Is All You Need' (2017).", "The attention matrix has  $O(n^2)$  complexity in sequence length  $n$ .", "BERT uses bidirectional masked language modelling for pre-training.", "Flash Attention reduces memory by tiling the attention computation.", ] response = co.rerank( model = "rerank-english-v3.0", query = query, documents = documents, top_n = 3, ) for r in response.results: print(f"[{r.relevance_score:.4f}] {documents[r.index]}")
```

Using HuggingFace bge-reranker

Open-source cross-encoder — runs locally with sentence-transformers:

```
from sentence_transformers import CrossEncoder model = CrossEncoder("BAAI/bge-reranker-v2-m3") query = "What causes transformer attention to be quadratic?" candidates = [ "The attention matrix has  $O(n^2)$  complexity in sequence length.", "BERT was pre-trained on masked language modelling.", "Flash Attention tiles the computation to save memory.", "Transformers were introduced in 2017.", ] pairs = [(query, doc) for doc in candidates] scores = model.predict(pairs) # returns numpy array of floats ranked = sorted(zip(scores, candidates), reverse=True) for score, doc in ranked: print(f"[{score:.4f}] {doc}")
```

Integrating with LangChain

Drop-in integration with an existing LangChain retriever:

```
from langchain.retrievers import ContextualCompressionRetriever from langchain_cohere import CohereRerank from langchain_community.vectorstores import Chroma # Assume vectorstore is already created base_retriever = vectorstore.as_retriever(search_kwargs={"k": 40}) # Wrap with Cohere Rerank compressor compressor = CohereRerank( model="rerank-english-v3.0", top_n=5 ) retriever = ContextualCompressionRetriever( base_compressor = compressor, base_retriever = base_retriever, ) # Use exactly like any other LangChain retriever docs = retriever.invoke("What causes attention to be quadratic?") for d in docs: print(d.page_content[:120])
```

09 Metrics & Evaluation

Use these metrics to measure the impact of adding reranking to your pipeline:

Metric	What it measures	Target
MRR@k (Mean Reciprocal Rank)	How high is the first relevant doc?	Higher is better
NDCG@k	Graded relevance of top-k ranked results	Higher is better
Recall@k	Fraction of relevant docs in top-k	Compare pre/post rerank
Precision@k	Fraction of top-k that are relevant	Usually improves with rerank
Faithfulness	Is the LLM answer grounded in context?	> 0.85 typical target
Answer Relevancy	Does the answer address the question?	> 0.80 typical target
Latency (p95)	End-to-end response time	< 2s for most apps

Quick eval setup with RAGAS:

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall
result = evaluate(
    dataset, # HuggingFace dataset with question/answer/contexts
    metrics=[faithfulness, answer_relevancy, context_recall],
)
print(result) # DataFrame with per-metric scores
```

10 Best Practices

Retrieve more, rerank down

Retrieve 40–100 candidates with vector search; rerank to top 5–10. Larger initial candidate pools give the reranker more to work with.

Match chunk size to your reranker's context window

Most cross-encoders handle 512 tokens. Jina Reranker v2 handles 8192. Ensure your chunks fit; truncation silently degrades quality.

Benchmark before choosing a reranker

BEIR is the standard benchmark. Top performers vary by domain — bge-reranker-v2-m3 is strong generally; domain-specific fine-tuning often outperforms all.

Cache reranker scores for repeated queries

Reranker calls are expensive. Use a query hash as a cache key; TTL of a few hours is safe for most knowledge bases.

Use late chunking or contextual chunking

Short isolated chunks lose surrounding context. Late chunking (embed at full doc, split later) or contextual compression (Anthropic technique) improves both retrieval and reranking quality.

Monitor latency separately for each stage

Track vector search latency, reranker latency, and LLM latency independently. Reranker is usually 100–500ms; spike detection is easier with separate metrics.

Reranking is one of the highest-ROI improvements you can make to a RAG pipeline. A two-stage retrieve-then-rerank architecture consistently outperforms naive top-k retrieval on both retrieval metrics and downstream answer quality.

Start with a hosted API (Cohere Rerank) for fast iteration, then evaluate open-source alternatives like bge-reranker for cost optimisation.